

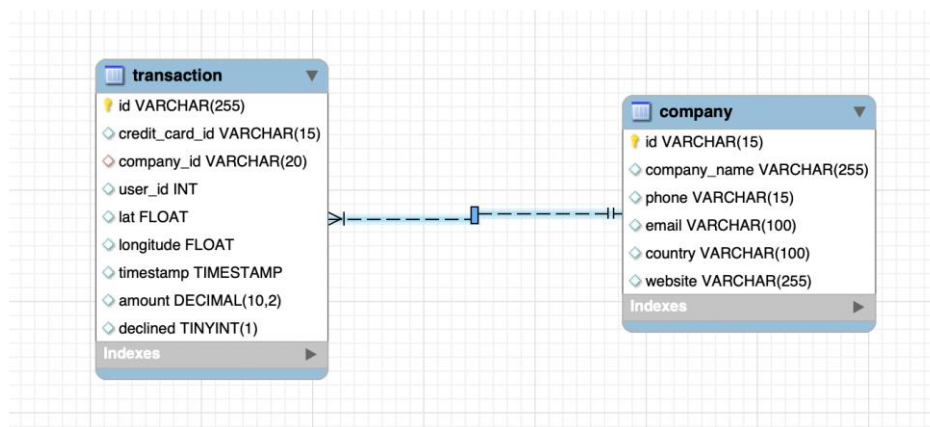
Tasca S2.01. Nocions bàsiques SQL

- Daniel Sánchez Peláez -

Nivell 1

- **Ex. 1.** A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Con el código facilitado se crean las dos tablas que se pueden observar en el siguiente diagrama:



La tabla *transaction* tiene las siguientes columnas:

- **id**: es de tipo texto y es la clave primaria (PK) de la tabla, representado por el símbolo del rayo amarillo. Permite una longitud máxima de 255 caracteres.
- **credit_card_id**: tipo texto con una longitud máxima permitida de 15 caracteres.
- **company_id**: tipo texto con una longitud máxima permitida de 20 caracteres. En rojo se indica que es una clave foránea (FK). En este caso solo puede ser de la tabla *company*.
- **user_id**: tipo nombre enter.
- **lat** i **longitude**: son columnas de nombres con decimales.
- **timestamp**: columna para almacenar fecha y hora, teniendo en cuenta la zona horaria, dicho de otro modo, hace una captura del momento, en lugar de capturar fecha y hora únicamente como haría DATETIME.
- **amount**: columna de nombres decimales donde se aceptan un máximo de 10 dígitos y 2 después del punto decimal. Es más preciso que float.

- declined: columna tipo tinyint. Este tipo de dato se usa para almacenar números pequeños. En este caso, solo admitirá 0 o 1, por lo tanto, emula una columna de tipo booleana, pero con 0 y 1's, en vez de True/False.

Respecto a la tabla *company* solo hay columnas tipo texto con diferentes tamaños máximos permitidos: id (15), esta es la PK, company_name (255), phone (15), email (100), country (100), website (255).

Por último, la relación entre las tablas es mediante la columna *companyid*, que es PK a la tabla *company* (*company.id*) y FK en la tabla *transaction* (*transaction.company_id*). La relación también se define como una relación de muchos (*transaction*) a uno (*company*), N:1.

- **Ex. 2. Utilitzant JOIN realitzaràs les següents consultes:**

- Llistat dels països que estan generant vendes.

Los países desde donde se hacen se hacen ventas son los que se pueden ver a continuación como resultado de la consulta:

```

1 • USE transactions;
2 • # Llistat dels països que estan generant vendes.
3 • SELECT DISTINCT(company.country)
4 • FROM transaction
5 • JOIN company ON transaction.company_id = company.id
6 • WHERE transaction.declined = 0;
7

```

country
Netherlands
Sweden
Ireland
United States
Belgium
Canada
Germany
Norway
France
Italy
United King...
New Zealand
China
Spain
Australia

Time	Action	Response	Duration / Fetch Time
40...	10:18:42 SELECT * FROM transactions.transaction	100000 row(s) returned	0.0014 sec / 0.100 sec
40...	11:23:56 USE transactions	0 row(s) affected	0.022 sec
40...	11:30:28 SELECT * FROM transactions.transaction	100000 row(s) returned	0.024 sec / 0.165 sec
40...	11:30:52 SELECT DISTINCT(company.country) FROM transaction JOIN company ON transaction.company_id = company.id WHERE transaction.declined = 0	Error Code: 1305. FUNCTION transactions.DISTINCT...	0.0081 sec
40...	11:32:02 SELECT DISTINCT(company.country) FROM transaction JOIN company ON transaction.company_id = company.id WHERE transaction.declined = 0	15 row(s) returned	0.168 sec / 0.000015...

La consulta se basa en juntar las dos tablas a partir de la columna *company_id* (o *company.id*), filtrar y seleccionar todo lo que es declined = 0, es decir, todas las transacciones que se han dado y que, por ende, no se han rechazado. Para poder hacer esta suposición doy por sentado que 0 es transacción aceptada y 1 es transacción rechazada. Finalmente, mediante DISTINCT selecciono las claves únicas de la columna country de la tabla *company*.

- Des de quants països es generen les vendes.

```

7 # Des de quants països es generen les vendes.
8 SELECT COUNT(DISTINCT company.country)
9 FROM transaction
10 JOIN company ON transaction.company_id = company.id
11 WHERE transaction.declined = 0;

```

Result Grid

COUNT(DISTINCT company.country)
15

Result 3

Action Output

Time	Action	Response	Duration / Fetch Time
40... 11:30:52	SELECT COUNT(DISTINCT(company.country)) FROM transaction JOIN company ON transaction.company_id = company.id WHERE transaction.declined = 0	Error Code: 1305. FUNCTION transactions.DISTINCT...	0.0081 sec
40... 11:32:02	SELECT COUNT(DISTINCT(company.country)) FROM transaction JOIN company ON transaction.company_id = company.id WHERE transaction.declined = 0	15 row(s) returned	0.169 sec / 0.000015...
40... 11:31:18	SELECT COUNT(DISTINCT(company.country)) FROM transaction JOIN company ON transaction.company_id = company.id WHERE transaction.declined = 0	Error Code: 1630. FUNCTION count.DISTINCT does n...	0.033 sec
40... 11:31:29	SELECT COUNT(DISTINCT(company.country)) FROM transaction JOIN company ON transaction.company_id = company.id WHERE transaction.declined = 0	1 row(s) returned	0.162 sec / 0.00054...
40... 11:32:42	SELECT COUNT(DISTINCT(company.country)) FROM transaction JOIN company ON transaction.company_id = company.id WHERE transaction.declined = 0	1 row(s) returned	0.176 sec / 0.000912...

Obtengo el número total de países distintos des de los que se realizan ventas usando la misma consulta de antes, pero añadiendo un COUNT, tal como se puede ver en el recorte.

- Identifica la companyia amb la mitjana més gran de vendes.

```

16 # Identifica la companyia amb la mitjana més gran de vendes.
17 SELECT company.company_name, AVG(transaction.amount) AS media
18 FROM transaction
19 JOIN company ON transaction.company_id = company.id
20 GROUP BY company.id
21 ORDER BY media DESC
22 LIMIT 1;

```

Result Grid

company_name	media
Ac Fermentum Incorporated	284.867160

Result 14

Action Output

Time	Action	Response	Duration / Fetch Time
18 09:43:57	SELECT company.company_name, AVG(transaction.amount) AS media FROM transaction JOIN company ON transaction.company_id = company.id GROUP B...	1 row(s) returned	0.233 sec / 0.000012...

- Ex. 3. Utilitzant només subconsultes (sense utilitzar JOIN):

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

```

20 # Exercici 3. Utilitzant nomes subconsultes (sense utilitzar JOIN):
21 # Mostra totes les transaccions realitzades per empreses d'Alemanya.
22 SELECT *
23 FROM transaction
24 WHERE company_id IN (
25     SELECT id
26     FROM company
27     WHERE country = "Germany");

```

Result Grid

id	credit_card...	company_id	user_id	lat	longitude	timestamp	amount	declined
001380D8-206D-4C03-94B7-63A267EDB984	CCS-4899	b-2222	318	41.3761	12.417	2020-03-25 10:43:43	106.36	0
0019C1B2-3B84-4D6C-B154-E2B3FEB0C8E9	CCS-5070	b-2222	489	41.3814	12.18176	2020-12-17 18:15:27	318.90	0
00201A11-2E82-44C4-941D-198FCDB77F0	CCU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CCS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CCS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0
00687139-4852-4FFA-5E73-520376F0A8A4	CCS-4870	b-2222	289	51.1966	10.4660	2019-03-09 19:37:49	524.84	0
007427A3-53E1-4937-876A-8355377259A4	CCS-3941	b-2222	1624	52.5018	-2.810936	2011-10-28 19:42:57	183.50	0

transaction 8

Action Output

Time	Action	Response	Duration / Fetch Time
40... 12:11:31	SELECT company_name FROM company WHERE country = "Germany"	8 row(s) returned	0.0024 sec / 0.00001...
40... 12:17:24	SELECT id, company_name, amount FROM transaction WHERE company_name IN (SELECT company_name FROM company WHERE country = "Germ...	Error Code: 1054. Unknown column 'company_name'...	0.0047 sec
40... 12:18:19	SELECT id, company_id, amount FROM transaction WHERE company_name IN (SELECT company_name FROM company WHERE country = "Germany")	Error Code: 1054. Unknown column 'company_name'...	0.0028 sec
40... 12:18:58	SELECT id, company_id, amount FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = "Germany")	13291 row(s) returned	0.0064 sec / 0.0067 sec
40... 12:19:31	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company WHERE country = "Germany")	13291 row(s) returned	0.0042 sec / 0.070 sec

- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

```

27 # Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
28 SELECT *
29 FROM company
30 WHERE id IN (
31     SELECT company_id
32     FROM transaction
33     WHERE amount > (SELECT AVG(amount)
34                     FROM transaction));
35

```

id	company_name	phone	email	country	website
b-2486	Clair Crax Associates	08 12 73 87 84	claxcrax@yahoo.com	Italy	https://ihsid.com/
b-2486	Tricount Oro Limited	01 78 18 81 44	velti.pelleresque@outlook.edu	Norway	https://mytimes.com/settings
b-2486	Nunc Sit Incorporated	07 28 42 63 63	jodee.ultraces@icloud.net	Norway	https://naver.com/en-ca
b-2486	Minevaust PSC	07 14 85 68 70	crax.soren.kronn@icloud.com	Ireland	https://yahoo.com/
b-2486	Meius Vitae Associates	08 28 44 40 66	kd@protonmail.com	Australia	https://google.com/
b-2502	Dolor Vitae Limited	06 53 80 43 60	purus.maecenas@yahoo.edu	France	https://whatsapp.com/user/110

company 11

Time	Action	Response	Duration / Fetch Time
40... 12:23:44	SELECT AVG(amount) FROM transaction	1 row(s) returned	0.050 sec / 0.000026...
40... 12:24:36	SELECT * FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction)	40722 row(s) returned	0.040 sec / 0.066 sec
40... 12:25:56	SELECT * FROM company WHERE SELECT company_id, amount FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction)	Error Code: 1064: You have an error in your SQL syntax; ...	0.00025 sec
40... 12:26:38	SELECT * FROM company WHERE id IN (SELECT company_id, amount FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction)	Error Code: 1241: Operand should contain 1 column(s)	0.0025 sec
40... 12:27:31	SELECT * FROM company WHERE id IN (SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction))	100 row(s) returned	0.083 sec / 0.00008...

- Eliminarian del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

```

34 # Eliminarian del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.
35 SELECT *
36 FROM company
37 WHERE id IN (
38     SELECT company_id
39     FROM transaction);
40

```

id	company_name	phone	email	country	website
b-2222	Az Fermentum Incorporated	08 88 88 88 33	donec.porrtibus@telus@yahoo.net	Germany	https://instagram.com/vite
b-2222	Magna A Nunc Industries	06 14 44 64 62	quis.donec.nish@icloud.org	Australia	https://whatsapp.com/group9
b-2230	Fusce Corp.	08 14 97 58 85	mauris@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.co.uk	Germany	https://corn.com/user/110
b-2238	Ante Incidunt Nec Foundation	06 23 04 89 55	sed.dictum.prim@outlook.ca	New Zealand	https://metrix.com/settings
b-2242	Donec Ltd	01 95 51 37 31	id.aculis@protonmail.co.uk	Norway	https://mytimes.com/user/110
b-2548	Quid Molest Ltd	09 69 64 79 48	id.aculis@icloud.net	Ireland	https://naver.com/

company 13

Time	Action	Response	Duration / Fetch Time
40... 12:25:56	SELECT * FROM company WHERE SELECT company_id, amount FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction)	Error Code: 1064: You have an error in your SQL synta...	0.00025 sec
40... 12:26:38	SELECT * FROM company WHERE id IN (SELECT company_id, amount FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction)	Error Code: 1241: Operand should contain 1 column(s)	0.0025 sec
40... 12:27:31	SELECT * FROM company WHERE id IN (SELECT company_id FROM transaction WHERE amount > (SELECT AVG(amount) FROM transaction)	100 row(s) returned	0.083 sec / 0.00008...
40... 12:30:29	SELECT company_id FROM transaction	100000 row(s) returned	0.0033 sec / 0.043 sec
40... 12:31:09	SELECT * FROM company WHERE id IN (SELECT company_id FROM transaction)	100 row(s) returned	0.0083 sec / 0.0000...

- **Ex. 4. La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda**

mostrar el diagrama i realitzar una breu descripció d'aquest.

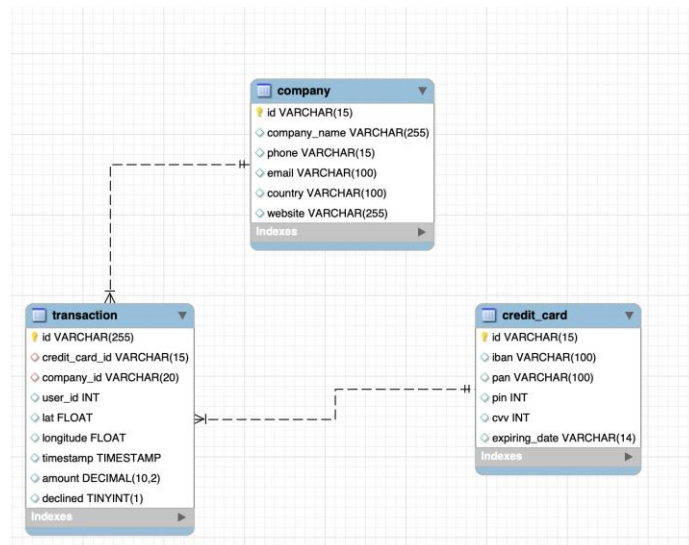
```
21 CREATE TABLE IF NOT EXISTS credit_card (  
22   id VARCHAR(15) PRIMARY KEY,  
23   iban VARCHAR(100),  
24   pan VARCHAR(100),  
25   pin INT,  
26   cvv INT,  
27   expiring_date VARCHAR(14)  
28 );  
29  
30 -- Creamos la tabla transaction  
31 CREATE TABLE IF NOT EXISTS transaction (  
32   id VARCHAR(255) PRIMARY KEY,  
33   credit_card_id VARCHAR(15) REFERENCES credit_card(id),  
34   company_id VARCHAR(20),  
35   user_id INT, # REFERENCES user(id), -- Queda comentat fins a crear aquesta taula.  
36   lat FLOAT,  
37   longitude FLOAT,  
38   timestamp TIMESTAMP,  
39   amount DECIMAL(10, 2),  
40   declined BOOLEAN,  
41   FOREIGN KEY (company_id) REFERENCES company(id)  
42 );  
43  
44  
45
```

Time	Action	Response	Duration / Fetch Time
13:10:06	USE transactions	0 row(s) affected	0.00027 sec
13:10:06	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(15), email VA...	0 row(s) affected, 1 warning(s): 1050 Table 'company'...	0.0013 sec
13:10:06	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(100), pan VARCHAR(100), pin INT, cvv IN...	0 row(s) affected, 1 warning(s): 1050 Table 'credit_ca...	0.00062 sec
13:10:06	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHAR(15) REFERENCES credit_card(id), com...	0 row(s) affected, 1 warning(s): 1050 Table 'transacti...	0.00046 sec
13:10:20	SELECT * FROM transactions.credit_card	5000 row(s) returned	0.0011 sec / 0.0088 s...

Genero una nueva tabla que pueda contener los datos facilitados en el archivo *N1-Ex.4_datos_introducir_credit.sql*. Para ello creo la tabla `credit_card` con las siguientes columnas:

- `id`: es la clave primaria, que a su vez se vinculará como clave foránea a `transaction` (`transaction.credit_card_id`).
- `iban` y `pan`: columnas tipo texto para datos de las tarjetas.
- `pin` y `cvv`: información de seguridad de las tarjetas en formato numérico (entero).
- `expiring_date`: en este caso, aunque por lógica la tipología de dato debería ser `DATE`, no es posible, ya que los datos se encuentran en formato `DD-MM-YY` (o similar). Se podría modificar el formato de los datos para que fuera `YYYY-MM-DD`, pero no lo hago para este ejercicio, sino que simplemente identifico la columna como texto.

El diagrama es el siguiente:



La nueva tabla se une a *transaction* (*transaction.credit_card_id*) mediante *id.credit_card*, tal y como se comenta en la explicación anterior. Con una relación de N:1 de *transaction* a *credit_card*.

- **Ex. 5. El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és:
TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.**

Datos antes del cambio:

```

39 # Exercici 5
40 SELECT *
41 FROM credit_card
42 WHERE id = "CcU-2938";
  
```

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	10/30/22

Datos después del cambio:

```

39 # Exercici 5
40 UPDATE credit_card
41 SET iban = "TR323456312213576817699999"
42 WHERE id = "CCU-2938";
43 SELECT *
44 FROM credit_card
45 WHERE id = "CCU-2938";

```

id	iban	pan	pin	cvv	expiring_date
CCU-2938	TR323456312213576817699999	5424465566813633	3257	984	10/30/22

credit_card 16

Time	Action	Response	Duration / Fetch Time
51... 13:10:20	SELECT * FROM transactions.credit_card	5000 row(s) returned	0.0011 sec / 0.0088 s...
51... 08:42:22	SELECT * FROM credit_card WHERE id = "CCU-2938"	1 row(s) returned	0.0047 sec / 0.00001...
51... 08:45:39	SELECT * FROM credit_card WHERE id = "CCU-2938"	1 row(s) returned	0.0011 sec / 0.00000...
51... 08:45:44	UPDATE credit_card SET iban = "TR323456312213576817699999" WHERE id = "CCU-2938"	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.014 sec
51... 08:45:46	SELECT * FROM credit_card WHERE id = "CCU-2938"	1 row(s) returned	0.00060 sec / 0.000...

- **Ex. 6. En la taula "transaction" ingressa una nova transacció amb la següent informació:**

Para resolver este ejercicio, antes de introducir los datos nuevos en la tabla *transaction* hay que introducir los campos que son clave foránea en *transaction* y clave primaria en *credit_card* (id) y *company* (id).

```

46 # Exercici 6
47 INSERT INTO company (id)
48 VALUES ("b-9999");
49 INSERT INTO credit_card (id)
50 VALUES ("CCU-9999");
51 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
52 VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "CCU-9999", "b-9999", 9999, 829,999, -117.999, 111.11, 0);
53
54

```

Time	Action	Response	Duration / Fetch Time
51... 15:25:58	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "CCU-9999", "b-9999", 9999, 829,999, -117.999, 111.11, 0);	Error Code: 1452. Cannot add or update a child row: a...	0.044 sec
51... 15:38:10	SELECT * FROM credit_card WHERE id = "CCU-9999"	0 row(s) returned	0.012 sec / 0.00030 s...
51... 09:40:53	INSERT INTO company (id) VALUES ("b-9999")	1 row(s) affected	0.046 sec
51... 09:41:44	INSERT INTO credit_card (id) VALUES ("CCU-9999")	1 row(s) affected	0.0037 sec
51... 09:41:47	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "CCU-9999", "b-9999", 9999, 829,999, -117.999, 111.11, 0);	1 row(s) affected	0.013 sec

- **Ex. 7. Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordra mostrar el canvi realitzat.**

```

53 # Exercici 7
54 ALTER TABLE credit_card
55 DROP COLUMN pan;
56

```

Time	Action	Response	Duration / Fetch Time
51... 15:38:10	SELECT * FROM credit_card WHERE id = "CCU-9999"	0 row(s) returned	0.012 sec / 0.00030 s...
51... 09:40:53	INSERT INTO company (id) VALUES ("b-9999")	1 row(s) affected	0.046 sec
51... 09:41:44	INSERT INTO credit_card (id) VALUES ("CCU-9999")	1 row(s) affected	0.0037 sec
51... 09:41:47	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "CCU-9999", "b-9999", 9999, 829,999, -117.999, 111.11, 0);	1 row(s) affected	0.013 sec
51... 09:55:05	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.074 sec

- **Ex. 8. Descarrega els arxius CSV que trobaràs a l'apartat de recursos. Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguem realitzar les següents consultes:**

La taula de products.csv l'utilitzarem més endavant.

Creo un nuevo database para todas estas tablas llamado trans_client, y genero una estructura de tabla para cada una de ellas. Después cargo los datos desde los CSV.

A la hora de crear las tablas, hago las columnas VARCHAR datatype ante el desconocimiento de los datos que me voy a encontrar, así como acepto NULLS por si alguna fila tuviera alguno. Si fuera necesario trabajar con los datos transformados a alguna tipología concreta, se podría modificar en el momento de necesitarlo. En casos donde el propio nombre de la columna identifique la tipología de los datos, sí que fuerzo ese tipo concreto.

Adjunto recorte de la creación y cargado de datos de una de las tablas, pero el procedimiento es el mismo para todas:

```
CREATE DATABASE trans_client;
USE trans_client;

59 CREATE TABLE IF NOT EXISTS american_users(
60     id VARCHAR(255) NULL,
61     name VARCHAR(255) NULL,
62     surname VARCHAR(255) NULL,
63     phone VARCHAR(255) NULL, # Utilizo VARCHAR porque no conozco los datos y no sé si se incluirán solo números o otros caracteres.
64     email VARCHAR(255) NULL,
65     birth_date VARCHAR(255) NULL,
66     country VARCHAR(255) NULL,
67     city VARCHAR(255) NULL,
68     postal_code VARCHAR(255) NULL,
69     address VARCHAR(255) NULL,
70     signup_date VARCHAR(255) NULL,
71     user_segment VARCHAR(255) NULL,
72     income_band VARCHAR(255) NULL);
73 LOAD DATA LOCAL INFILE '/Users/daniel/Library/CloudStorage/OneDrive-Personal/ITAcademy/Especializació/SOL/S2/N1-Ex-8_american_users.csv'
74 INTO TABLE american_users
75 FIELDS TERMINATED BY ','
76 OPTIONALLY ENCLOSED BY '"'
77 IGNORE 1 ROWS;
78
```

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band
8	Nissim	Franks	(850) 157-3489	egestas.aliquam.kingla@google.ca	Aug 1 1989	United States	New York	10001	Ap #251-7144 Integer St.	2013-03-18	young_professional	
9	Marvin	Mcain	(560) 883-7184	aliquam.nisl@outlook.com	Jan 24 1987	United States	San Antonio	78201	647-3080 Latius St.	2014-02-15	young_professional	
10	Robert	McCarthy	(324) 740-6171	veniam.tam@protonmail.com	Apr 30 1984	United States	San Jose	95101	P.O. Box 779	2013-10-19	premium	

american_users 1

Time	Action	Response	Duration / Fetch Time
13 11:17:38	LOAD DATA LOCAL INFILE '/Users/daniel/Desktop/N1-Ex-8_american_users.csv' INTO TABLE american_users FIELDS TERMINATED BY ',' IGNORE 1 ROWS	1010 row(s) affected, 1010 warning(s); 1262 Row 1 wa...	2.523 sec
14 11:20:16	LOAD DATA LOCAL INFILE '/Users/daniel/Library/CloudStorage/OneDrive-Personal/ITAcademy/Especializació/SOL/S2/N1-Ex-8_american_users.csv' INTO TA...	1010 row(s) affected, 1010 warning(s); 1262 Row 1 wa...	0.028 sec
15 11:21:29	LOAD DATA LOCAL INFILE '/Users/daniel/Library/CloudStorage/OneDrive-Personal/ITAcademy/Especializació/SOL/S2/N1-Ex-8_american_users.csv' INTO TA...	1010 row(s) affected; Records: 1010 Deleted: 0 Skips...	0.034 sec
16 11:22:00	SELECT * FROM american_users LIMIT 10	10 row(s) returned	0.0012 sec / 0.00062...

A la hora de crear la tabla de transacciones sí que he comprobado previamente que tipo de dato había en las columnas relacionadas con importes, ya que son necesarias en formato numérico para contestar la última pregunta del sprint.

```
169 CREATE TABLE IF NOT EXISTS transactions(
170     id VARCHAR(255) PRIMARY KEY,
171     card_id VARCHAR(255),
172     business_id VARCHAR(255),
173     timestamp VARCHAR(255) NULL,
174     amount DECIMAL(10,2) NULL,
175     declined VARCHAR(255) NULL,
176     product_ids VARCHAR(255) NULL,
177     user_id VARCHAR(255) NULL,
178     lat VARCHAR(255) NULL,
179     longitude VARCHAR(255) NULL,
180     discount_amount DECIMAL(10,2) NULL,
181     tax_amount DECIMAL(10,2) NULL,
182     shipping_amount DECIMAL(10,2) NULL,
183     channel VARCHAR(255) NULL,
184     campaign_id VARCHAR(255) NULL,
185     device_type VARCHAR(255) NULL,
186     is_international VARCHAR(255) NULL,
187     decline_reason VARCHAR(255) NULL,
188     distance_km VARCHAR(255) NULL,
189
190     FOREIGN KEY (card_id) REFERENCES credit_cards(id),
191     FOREIGN KEY (business_id) REFERENCES companies(company_id));
```

Al crear la tabla también genero la conexión con otras tablas mediante claves foráneas.

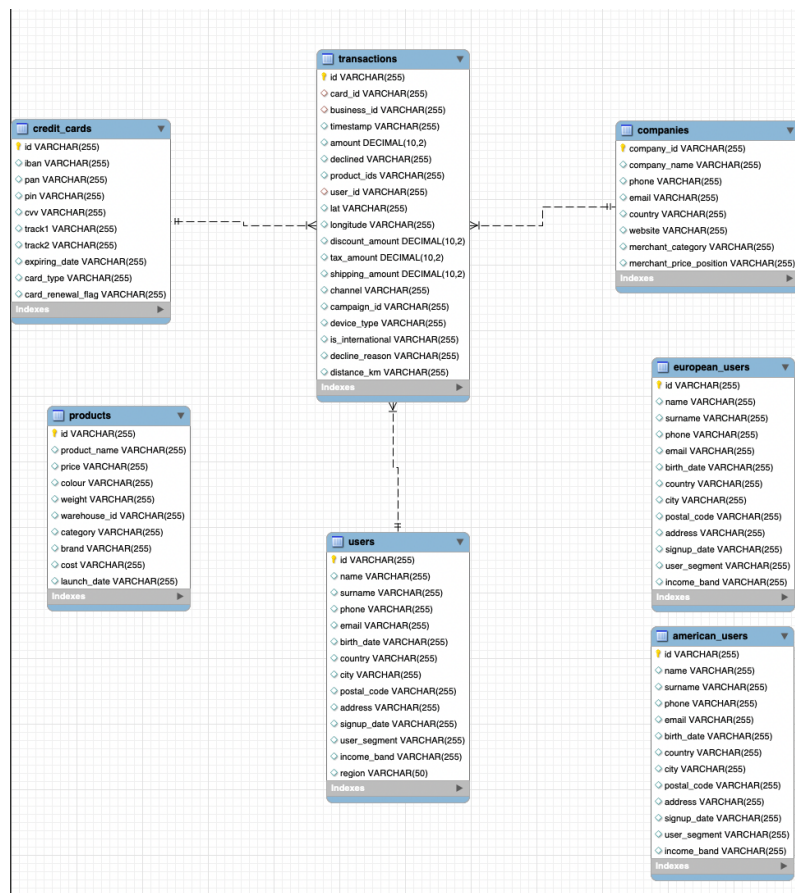
Por último, para poder crear el diagrama de estrella que se pide, genero una nueva tabla: *clients*. Esta tabla une las dos tablas de clientes, creando en el proceso una nueva columna que identifica estos clientes según la región (ergo, según tabla de origen).

```
CREATE TABLE IF NOT EXISTS users (
  id VARCHAR (255) PRIMARY KEY,
  name VARCHAR (255) NULL,
  surname VARCHAR (255) NULL,
  phone VARCHAR (255) NULL,
  email VARCHAR (255) NULL,
  birth_date VARCHAR (255) NULL,
  country VARCHAR (255) NULL,
  city VARCHAR (255) NULL,
  postal_code VARCHAR (255) NULL,
  address VARCHAR (255) NULL,
  signup_date VARCHAR (255) NULL,
  user_segment VARCHAR (255) NULL,
  income_band VARCHAR (255) NULL,
  region VARCHAR (50) NULL
);

INSERT INTO users (id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, region)
SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, 'europe' FROM european_users
UNION
SELECT id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band, 'america' FROM american_users;

ALTER TABLE transactions
ADD CONSTRAINT fk_transactions_users
FOREIGN KEY (user_id) REFERENCES users(id);
```

El esquema queda de la siguiente manera, teniendo en cuenta que la tabla de productos queda pendiente de usar más adelante.



- **Ex. 9. Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.**

```

230 # Exercici 9
231 SELECT *
232 FROM users
233 WHERE id IN (
234     SELECT user_id
235     FROM transactions
236     GROUP BY user_id
237     HAVING COUNT(DISTINCT id) > 80);

```

Result Grid

id	name	surname	phone	email	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band	region
185	Molly	Gilliam	0800 120 8023	donec@outlook.co.uk	Dec 21, 1993	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 MI Rd.	2013-08-11	frequent shopper	high	europa
289	Dewig	Hwcuru	+99-309-8797	dewig.hwcuru@example.com	Aug 20, 1976	Germany	Stuttgart	70173	82 Hwcuru Street	2013-12-11	occasional	low	europa
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa	16100	53 Astuw Street	2013-05-29	frequent shopper	very_high	europa
454	Sfzzoh	Xgvfndxs	+58-495-3945	sfzzoh.xgvfndxs@example.com	Aug 26, 1962	Poland	Gdansk	80-001	52 Xgvfndxs Street	2014-08-30	established	medium	europa

users 15

Action Output

	Time	Action	Response	Duration / Fetch Time
21	10:56:10	SELECT * FROM users WHERE id IN (SELECT user_id FROM transactions GROUP BY user_id HAVING COUNT(DISTINCT id) > 80)	4 row(s) returned	0.162 sec / 0.000016...

- Ex. 10. Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```

244 # Exercici 10
245 SELECT AVG(transactions.amount), iban
246 FROM credit_cards
247 JOIN transactions ON transactions.card_id = credit_cards.id
248 JOIN companies ON companies.company_id = transactions.business_id
249 WHERE companies.company_name = "Donec Ltd"
250 GROUP BY iban;

```

Result Grid

AVG(transactions.amou...)	iban
361.010000	CA901355773586771626108990
142.580000	US729212642816955651428791
248.900000	US89942680323093276816251
154.010000	US581097432668325541142297
250.510000	CA441226267105857473570574
203.950000	CA332171962352073015086775
438.160000	US985359628701198880628720

Result 10

Action Output

	Time	Action	Response	Duration / Fetch Time
35	10:12:45	SELECT AVG(transactions.amount), iban FROM credit_cards JOIN transactions ON transactions.card_id = credit_cards.id JOIN companies ON companies.co...	5000 row(s) returned	0.423 sec / 0.0017 sec
36	10:14:10	SELECT AVG(transactions.amount), iban FROM credit_cards JOIN transactions ON transactions.card_id = credit_cards.id JOIN companies ON companies.co...	371 row(s) returned	0.017 sec / 0.000074...
37	10:14:20	SELECT AVG(transactions.amount), iban, company_name FROM credit_cards JOIN transactions ON transactions.card_id = credit_cards.id JOIN companies O...	371 row(s) returned	0.0074 sec / 0.00007...
38	10:14:27	SELECT AVG(transactions.amount), iban FROM credit_cards JOIN transactions ON transactions.card_id = credit_cards.id JOIN companies ON companies.co...	371 row(s) returned	0.0069 sec / 0.0000...

Nivell 2:

- Ex. 1. Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```

253 -- Nivell 2
254 # Exercici 1
255 SELECT DATE(timestamp), SUM(amount) AS ventas
256 FROM transactions
257 GROUP BY DATE(timestamp)
258 ORDER BY ventas DESC;
259
260
261
262

```

100% 19:249

Result Grid Filter Rows: Search Export: Fetch rows:

DATE(timestamp)	ventas
2019-11-18	15840.24
2023-02-20	15728.40
2022-12-13	15129.20
2019-03-18	14896.47
2017-12-20	14360.96
2021-10-27	13417.55
2023-07-17	13399.86

Result 14 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
41	10:26:04	SELECT DATE(timestamp), SUM(amount) FROM transactions GROUP BY DATE(timestamp)	3653 row(s) returned	0.090 sec / 0.0011 sec
42	10:26:32	SELECT DATE(timestamp), SUM(amount) AS ventas FROM transactions GROUP BY DATE(timestamp) ORDER BY ventas	3653 row(s) returned	0.079 sec / 0.00036...
43	10:26:40	SELECT DATE(timestamp), SUM(amount) AS ventas FROM transactions GROUP BY DATE(timestamp) ORDER BY ventas DESC FALSE	Error Code: 1064. You have an error in your SQL synta...	0.00042 sec
44	10:26:46	SELECT DATE(timestamp), SUM(amount) AS ventas FROM transactions GROUP BY DATE(timestamp) ORDER BY ventas DESC	3653 row(s) returned	0.078 sec / 0.0012 sec

- Ex. 2. Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```

260 # Exercici 2
261 SELECT companies.company_name, companies.phone, companies.country, DATE(transactions.timestamp), transactions.amount
262 FROM transactions
263 JOIN companies ON transactions.business_id = companies.company_id
264 WHERE DATE(transactions.timestamp) IN ("2015-04-29", "2018-07-20", "2024-03-13")
265 AND transactions.amount >350
266 AND transactions.amount <400
267 ORDER BY transactions.amount DESC;
268
269

```

100% 7:254

Result Grid Filter Rows: Search Export:

company_name	phone	country	DATE(transactions.timestamp)	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 788 0126	Norway	2018-07-20	358.64
Natus Et Malesuada Ltd	+22 918 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3138	Netherlands	2024-03-13	350.71

Result 15 Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
51	10:40:08	SELECT companies.company_name, companies.phone, companies.country, DATE(transactions.timestamp), SUM(transactions.amount) FROM transactions J...	Error Code: 1055. Expression #2 of SELECT list is not...	0.00072 sec
52	10:40:47	SELECT companies.company_name, companies.phone, companies.country, DATE(transactions.timestamp), SUM(transactions.amount) FROM transactions J...	Error Code: 1140. In aggregated query without GROU...	0.00085 sec
53	10:41:08	SELECT companies.company_name, companies.phone, companies.country, DATE(transactions.timestamp), SUM(transactions.amount) FROM transactions J...	Error Code: 1111. Invalid use of group function	0.00052 sec
54	10:44:38	SELECT companies.company_name, companies.phone, companies.country, DATE(transactions.timestamp), transactions.amount FROM transactions JOIN co...	6 row(s) returned	0.083 sec / 0.00045...

- Ex. 3. Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

```

268
269 # Exercici 3
270 SELECT companies.company_name, COUNT(DISTINCT transactions.id) AS recuento,
271 CASE
272   WHEN COUNT(DISTINCT transactions.id) >= 400 THEN "Igual o més de 400 transaccions realitzades"
273   ELSE "Menys de 400 transaccions"
274 END
275 FROM transactions
276 JOIN companies ON transactions.business_id = companies.company_id
277 GROUP BY companies.company_name
278 ORDER BY recuento DESC;
279
280

```

company_name	recuento	CASE WHEN COUNT(DISTINCT transactions.id)...
Ac Fermentum Incorporated	2401	Igual o més de 400 transaccions realitzades
Nunc Interdum Incorporated	1599	Igual o més de 400 transaccions realitzades
Donec Fringilla PC	1593	Igual o més de 400 transaccions realitzades
Mauris Institute	1586	Igual o més de 400 transaccions realitzades
Putrum Non Inc.	1585	Igual o més de 400 transaccions realitzades
Aliquet Vel Vulputate Incorporated	1583	Igual o més de 400 transaccions realitzades
Aliquam PC	1576	Igual o més de 400 transaccions realitzades

Time	Action	Response	Duration / Fetch Time
10:41:08	SELECT companies.company_name, COUNT(DISTINCT transactions.id) AS recuento, CASE WHEN COUNT(DISTINCT transactions.id) >= 400 THEN "Igual o més de 400 transaccions realitzades" ELSE "Menys de 400 transaccions" END FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name ORDER BY recuento DESC;	Error Code: 1111. Invalid use of group function	0.00062 sec
10:44:36	SELECT companies.company_name, COUNT(DISTINCT transactions.id) AS recuento, CASE WHEN COUNT(DISTINCT transactions.id) >= 400 THEN "Igual o més de 400 transaccions realitzades" ELSE "Menys de 400 transaccions" END FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name ORDER BY recuento DESC;	6 row(s) returned	0.083 sec / 0.00045...
11:02:37	SELECT companies.company_name, COUNT(DISTINCT transactions.id) AS recuento, CASE WHEN COUNT(DISTINCT transactions.id) >= 400 THEN "Igual o més de 400 transaccions realitzades" ELSE "Menys de 400 transaccions" END FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name ORDER BY recuento DESC;	Error Code: 1140. In aggregated query without GROUP BY	0.042 sec
11:03:22	SELECT companies.company_name, COUNT(DISTINCT transactions.id) AS recuento, CASE WHEN COUNT(DISTINCT transactions.id) >= 400 THEN "Igual o més de 400 transaccions realitzades" ELSE "Menys de 400 transaccions" END FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name ORDER BY recuento DESC;	100 row(s) returned	0.146 sec / 0.00030...

- **Ex. 4. Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.**

```

279
280 # Exercici 4
281 -- Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.
282 DELETE FROM transactions
283 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
284

```

Time	Action	Response	Duration / Fetch Time
11:02:37	SELECT companies.company_name, COUNT(DISTINCT transactions.id) AS recuento, CASE WHEN COUNT(DISTINCT transactions.id) >= 400 THEN "Igual o més de 400 transaccions realitzades" ELSE "Menys de 400 transaccions" END FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name ORDER BY recuento DESC;	Error Code: 1140. In aggregated query without GROUP BY	0.042 sec
11:03:22	SELECT companies.company_name, COUNT(DISTINCT transactions.id) AS recuento, CASE WHEN COUNT(DISTINCT transactions.id) >= 400 THEN "Igual o més de 400 transaccions realitzades" ELSE "Menys de 400 transaccions" END FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name ORDER BY recuento DESC;	100 row(s) returned	0.146 sec / 0.00030...
11:10:44	SELECT * FROM transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";	1 row(s) returned	0.039 sec / 0.00014...
11:14:35	DELETE FROM transactions WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";	1 row(s) affected	0.021 sec

- **Ex. 5. La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.**

```

284
285 # Exercici 5
286 CREATE VIEW VistaMarketing AS
287 SELECT companies.company_name, companies.phone, companies.country, AVG(transactions.amount) media_trans
288 FROM transactions
289 JOIN companies ON transactions.business_id = companies.company_id
290 GROUP BY companies.company_name, companies.phone, companies.country;
291
292 SELECT *
293 FROM VistaMarketing
294 ORDER BY media_trans;
295

```

company_name	phone	country	media_trans
Etiam Lacreet Associates	+39 253 349 8865	Canada	260.037273
Fringilla LLC	+1 152 784 1716	New Zealand	260.422220
Nunc Sit Incorporated	+62 449 706 4735	Norway	260.666958
Sapient Nunc Pulvinar LLP	+79 745 217 3551	New Zealand	260.682949
Arcu LLP	+83 776 813 5178	Norway	260.987136
Odio Cui Institute	+37 104 950 8546	New Zealand	261.589851
Pede Dum Ltd	+58 625 788 6126	Norway	262.931542

Time	Action	Response	Duration / Fetch Time
11:36:57	CREATE VIEW VistaMarketing AS SELECT companies.company_name, companies.phone, companies.country, AVG(transactions.amount) media_trans FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name, companies.phone, companies.country;	Error Code: 1050. Table 'VistaMarketing' already exists	0.00062 sec
11:37:11	DROP VIEW 'trans_clients', 'vistamarketing';	0 row(s) affected	0.0071 sec
11:37:15	CREATE VIEW VistaMarketing AS SELECT companies.company_name, companies.phone, companies.country, AVG(transactions.amount) media_trans FROM transactions JOIN companies ON transactions.business_id = companies.company_id GROUP BY companies.company_name, companies.phone, companies.country;	0 row(s) affected	0.0034 sec
11:37:22	SELECT * FROM VistaMarketing ORDER BY media_trans;	100 row(s) returned	0.292 sec / 0.000075...

Nivell 3

- **Ex. 1.** Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

```
299 CREATE TABLE targetes_actives AS
300 WITH ultimes AS (
301     SELECT
302         transactions.card_id,
303         transactions.declined,
304         ROW_NUMBER() OVER (
305             PARTITION BY transactions.card_id
306             ORDER BY transactions.timestamp DESC
307         ) AS tt_ultimes
308     FROM transactions
309 )
310 SELECT
311     card_id,
312     CASE
313         WHEN COUNT(*) = 3
314             AND SUM(ultimes.declined = 1) = 3
315         THEN 'inactiva'
316         ELSE 'activa'
317     END AS estat
318 FROM ultimes
319 WHERE tt_ultimes <= 3
320 GROUP BY card_id;
```

- **Quantes targetes estan actives?**

Hay activas 5.000 tarjetas.

```
313 SELECT COUNT(*)
314 FROM targetes_actives
315 WHERE estat = 'activa';
```

COUNT(*)
5000

Result 17

Time	Action	Response	Duration / Fetch Time
26 11:22:16	SELECT COUNT(*) FROM targetes_actives WHERE estat = 'activa'	1 row(s) returned	0.0090 sec / 0.00000...

- **Ex. 2.** Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Creo la tabla intermedia que se pide en el enunciado con el siguiente código:

```
1
2 # Ejercicio 2
3 CREATE TABLE IF NOT EXISTS product_transfer(
4     id INT AUTO_INCREMENT PRIMARY KEY,
5     product_id VARCHAR (255),
6     trans_id VARCHAR (255),
7     FOREIGN KEY (product_id) REFERENCES products(id),
8     FOREIGN KEY (trans_id) REFERENCES transactions(id));
9
```

```

30 INSERT INTO product_transfer (product_id, trans_id)
31 SELECT transactions_json.product_id, transactions.id
32 FROM transactions
33 JOIN JSON_TABLE(
34     CONCAT(
35         '["', REPLACE(transactions.product_ids, ',', '",""), ""]',
36         '["$"]' COLUMNS (product_id VARCHAR(255) PATH '$') AS transactions_json;
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

- Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```

278 SELECT count(product_transfer.product_id), product_name
279 FROM product_transfer
280 WHERE products ON products.id = product_transfer.product_id
281 GROUP BY product_transfer.product_id
282 ORDER BY count(product_transfer.product_id) DESC;
283
284
285
286

```

100% 49,342

Result Grid

Filter Rows

Export

COUNT(product_transfer.product_id)	product_name
2054	Overlands the dust
2055	Truly overland fairly
2056	Earl Overland
2057	One Earl Overland
2058	Overland (Pm)
2059	Overland (Pm)
2060	Earl Overland
2061	Earl Overland
2062	Overland (Pm)
2063	Overland (Pm)
2064	Overland (Pm)
2065	Overland (Pm)
2066	Overland (Pm)
2067	Overland (Pm)
2068	Overland (Pm)
2069	Overland (Pm)
2070	Overland (Pm)
2071	Overland (Pm)
2072	Overland (Pm)
2073	Overland (Pm)
2074	Overland (Pm)
2075	Overland (Pm)
2076	Overland (Pm)
2077	Overland (Pm)
2078	Overland (Pm)
2079	Overland (Pm)
2080	Overland (Pm)
2081	Overland (Pm)
2082	Overland (Pm)
2083	Overland (Pm)
2084	Overland (Pm)
2085	Overland (Pm)
2086	Overland (Pm)
2087	Overland (Pm)
2088	Overland (Pm)
2089	Overland (Pm)
2090	Overland (Pm)
2091	Overland (Pm)
2092	Overland (Pm)
2093	Overland (Pm)
2094	Overland (Pm)
2095	Overland (Pm)
2096	Overland (Pm)
2097	Overland (Pm)
2098	Overland (Pm)
2099	Overland (Pm)
2100	Overland (Pm)
2101	Overland (Pm)
2102	Overland (Pm)
2103	Overland (Pm)
2104	Overland (Pm)
2105	Overland (Pm)
2106	Overland (Pm)
2107	Overland (Pm)
2108	Overland (Pm)
2109	Overland (Pm)
2110	Overland (Pm)
2111	Overland (Pm)
2112	Overland (Pm)
2113	Overland (Pm)
2114	Overland (Pm)
2115	Overland (Pm)
2116	Overland (Pm)
2117	Overland (Pm)
2118	Overland (Pm)
2119	Overland (Pm)
2120	Overland (Pm)
2121	Overland (Pm)
2122	Overland (Pm)
2123	Overland (Pm)
2124	Overland (Pm)
2125	Overland (Pm)
2126	Overland (Pm)
2127	Overland (Pm)
2128	Overland (Pm)
2129	Overland (Pm)
2130	Overland (Pm)
2131	Overland (Pm)
2132	Overland (Pm)
2133	Overland (Pm)
2134	Overland (Pm)
2135	Overland (Pm)
2136	Overland (Pm)
2137	Overland (Pm)
2138	Overland (Pm)
2139	Overland (Pm)
2140	Overland (Pm)
2141	Overland (Pm)
2142	Overland (Pm)
2143	Overland (Pm)
2144	Overland (Pm)
2145	Overland (Pm)
2146	Overland (Pm)
2147	Overland (Pm)
2148	Overland (Pm)
2149	Overland (Pm)
2150	Overland (Pm)
2151	Overland (Pm)
2152	Overland (Pm)
2153	Overland (Pm)
2154	Overland (Pm)
2155	Overland (Pm)
2156	Overland (Pm)
2157	Overland (Pm)
2158	Overland (Pm)
2159	Overland (Pm)
2160	Overland (Pm)
2161	Overland (Pm)
2162	Overland (Pm)
2163	Overland (Pm)
2164	Overland (Pm)
2165	Overland (Pm)
2166	Overland (Pm)
2167	Overland (Pm)
2168	Overland (Pm)
2169	Overland (Pm)
2170	Overland (Pm)
2171	Overland (Pm)
2172	Overland (Pm)
2173	Overland (Pm)
2174	Overland (Pm)
2175	Overland (Pm)
2176	Overland (Pm)
2177	Overland (Pm)
2178	Overland (Pm)
2179	Overland (Pm)
2180	Overland (Pm)
2181	Overland (Pm)
2182	Overland (Pm)
2183	Overland (Pm)
2184	Overland (Pm)
2185	Overland (Pm)
2186	Overland (Pm)
2187	Overland (Pm)
2188	Overland (Pm)
2189	Overland (Pm)
2190	Overland (Pm)
2191	Overland (Pm)
2192	Overland (Pm)
2193	Overland (Pm)
2194	Overland (Pm)
2195	Overland (Pm)
2196	Overland (Pm)
2197	Overland (Pm)
2198	Overland (Pm)
2199	Overland (Pm)
2200	Overland (Pm)
2201	Overland (Pm)
2202	Overland (Pm)
2203	Overland (Pm)
2204	Overland (Pm)
2205	Overland (Pm)
2206	Overland (Pm)
2207	Overland (Pm)
2208	Overland (Pm)
2209	Overland (Pm)
2210	Overland (Pm)
2211	Overland (Pm)
2212	Overland (Pm)
2213	Overland (Pm)
2214	Overland (Pm)
2215	Overland (Pm)
2216	Overland (Pm)
2217	Overland (Pm)
2218	Overland (Pm)
2219	Overland (Pm)
2220	Overland (Pm)
2221	Overland (Pm)
2222	Overland (Pm)
2223	Overland (Pm)
2224	Overland (Pm)
2225	Overland (Pm)
2226	Overland (Pm)
2227	Overland (Pm)
2228	Overland (Pm)
2229	Overland (Pm)
2230	Overland (Pm)
2231	Overland (Pm)
2232	Overland (Pm)
2233	Overland (Pm)
2234	Overland (Pm)
2235	Overland (Pm)
2236	Overland (Pm)
2237	Overland (Pm)
2238	Overland (Pm)
2239	Overland (Pm)
2240	Overland (Pm)
2241	Overland (Pm)
2242	Overland (Pm)
2243	Overland (Pm)
2244	Overland (Pm)
2245	Overland (Pm)
2246	Overland (Pm)
2247	Overland (Pm)
2248	Overland (Pm)
2249	Overland (Pm)
2250	Overland (Pm)
2251	Overland (Pm)
2252	Overland (Pm)
2253	Overland (Pm)
2254	Overland (Pm)
2255	Overland (Pm)
2256	Overland (Pm)
2257	Overland (Pm)
2258	Overland (Pm)
2259	Overland (Pm)
2260	Overland (Pm)
2261	Overland (Pm)
2262	Overland (Pm)
2263	Overland (Pm)
2264	Overland (Pm)
2265	Overland (Pm)
2266	Overland (Pm)
2267	Overland (Pm)
2268	Overland (Pm)
2269	Overland (Pm)
2270	Overland (Pm)
2271	Overland (Pm)
2272	Overland (Pm)
2273	Overland (Pm)
2274	Overland (Pm)
2275	Overland (Pm)
2276	Overland (Pm)
2277	Overland (Pm)
2278	Overland (Pm)
2279	Overland (Pm)
2280	Overland (Pm)
2281	Overland (Pm)
2282	Overland (Pm)
2283	Overland (Pm)
2284	Overland (Pm)
2285	Overland (Pm)
2286	Overland (Pm)
2287	Overland (Pm)
2288	Overland (Pm)
2289	Overland (Pm)
2290	Overland (Pm)
2291	Overland (Pm)
2292	Overland (Pm)
2293	Overland (Pm)
2294	Overland (Pm)
2295	Overland (Pm)
2296	Overland (Pm)
2297	Overland (Pm)
2298	Overland (Pm)
2299	Overland (Pm)
2300	Overland (Pm)
2301	Overland (Pm)
2302	Overland (Pm)
2303	Overland (Pm)
2304	Overland (Pm)
2305	Overland (Pm)
2306	Overland (Pm)
2307	Overland (Pm)
2308	Overland (Pm)
2309	Overland (Pm)
2310	Overland (Pm)
2311	Overland (Pm)
2312	Overland (Pm)
2313	Overland (Pm)
2314	Overland (Pm)
2315	Overland (Pm)
2316	Overland (Pm)
2317	Overland (Pm)
2318	Overland (Pm)
2319	Overland (Pm)
2320	Overland (Pm)
2321	Overland (Pm)
2322	Overland (Pm)
2323	Overland (Pm)
2324	Overland (Pm)
2325	Overland (Pm)
2326	Overland (Pm)
2327	Overland (Pm)
2328	Overland (Pm)
2329	Overland (Pm)
2330	Overland (Pm)
2331	Overland (Pm)
2332	Overland (Pm)
2333	Overland (Pm)
2334	Overland (Pm)
2335	Overland (Pm)
2336	Overland (Pm)
2337	Overland (Pm)
2338	Overland (Pm)
2339	Overland (Pm)
2340	Overland (Pm)
2341	Overland (Pm)
2342	Overland (Pm)
2343	Overland (Pm)
2344	Overland (Pm)
2345	Overland (Pm)
2346	Overland (Pm)
2347	Overland (Pm)
2348	Overland (Pm)
2349	Overland (Pm)
2350	Overland (Pm)
2351	Overland (Pm)
2352	Overland (Pm)
2353	Overland (Pm)
2354	Overland (Pm)
2355	Overland (Pm)
2356	Overland (Pm)
2357	Overland (Pm)
2358	Overland (Pm)
2359	Overland (Pm)
2360	Overland (Pm)
2361	Overland (Pm)
2362	Overland (Pm)
2363	Overland (Pm)
2364	Overland (Pm)
2365	Overland (Pm)
2366	Overland (Pm)
2367	Overland (Pm)
2368	Overland (Pm)
2369	Overland (Pm)
2370	Overland (Pm)
2371	Overland (Pm)
2372	Overland (Pm)
2373	Overland (Pm)
2374	Overland (Pm)
2375	Overland (Pm)
2376	Overland (Pm)
2377	Overland (Pm)
2378	Overland (Pm)
2379	Overland (Pm)
2380	Overland (Pm)
2381	Overland (Pm)
2382	Overland (Pm)
2383	Overland (Pm)
2384	Overland (Pm)
2385	Overland (Pm)
2386	Overland (Pm)
2387	Overland (Pm)
2388	Overland (Pm)
2389	Overland (Pm)
2390	Overland (Pm)
2391	Overland (Pm)
2392	Overland (Pm)
2393	Overland (Pm)
2394	Overland (Pm)
2395	Overland (Pm)
2396	Overland (Pm)
2397	Overland (Pm)
2398	Overland (Pm)
2399	Overland (Pm)
2400	Overland (Pm)
2401	Overland (Pm)
2402	Overland (Pm)
2403	Overland (Pm)
2404	Overland (Pm)
2405	Overland (Pm)
2406	Overland (Pm)
2407	Overland (Pm)
2408	Overland (Pm)
2409	Overland (Pm)
2410	Overland (Pm)
2411	Overland (Pm)
2412	Overland (Pm)
2413	Overland (Pm)
2414	Overland (Pm)
2415	Overland (Pm)
2416	Overland (Pm)
2417	Overland (Pm)
2418	Overland (Pm)
2419	Overland (Pm)
2420	Overland (Pm)
2421	Overland (Pm)
2422	Overland (Pm)
2423	Overland (Pm)
2424	Overland (Pm)
2425	Overland (Pm)
2426	Overland (Pm)
2427	Overland (Pm)
2428	Overland (Pm)
2429	Overland (Pm)
2430	Overland (Pm)
2431	Overland (Pm)
2432	Overland (Pm)
2433	Overland (Pm)
2434	Overland (Pm)
2435	Overland (Pm)
2436	Overland (Pm)
2437	Overland (Pm)
2438	Overland (Pm)
2439	Overland (Pm)
2440	Overland (Pm)
2441	Overland (Pm)
2442	Overland (Pm)
2443	Overland (Pm)
2444	Overland (Pm)
2445	Overland (Pm)
2446	Overland (Pm)
2447	Overland (Pm)
2448	Overland (Pm)
2449	Overland (Pm)
2450	Overland (Pm)
2451	Overland (Pm)
2452	Overland (Pm)
2453	Overland (Pm)
2454	Overland (Pm)
2455	Overland (Pm)
2456	Overland (Pm)
2457	Overland (Pm)
2458	Overland (Pm)
2459	Overland (Pm)
2460	Overland (Pm)
2461	Overland (Pm)
2462	Overland (Pm)
2463	Overland (Pm)
2464	Overland (Pm)
2465	Overland (Pm)
2466	Overland (Pm)
2467	Overland (Pm)
2468	Overland (Pm)
2469	Overland (Pm)
2470	Overland (Pm)
2471	Overland (Pm)
2472	Overland (Pm)
2473	Overland (Pm)
2474	Overland (Pm)
2475	Overland (Pm)
2476	Overland (Pm)
2477	Overland (Pm)
2478	Overland (Pm)
2479	Overland (Pm)
2480	Overland (Pm)
2481	Overland (Pm)
2482	Overland (Pm)
2483	Overland (Pm)
2484	Overland (Pm)
2485	Overland (Pm)
2486	Overland (Pm)
2487	Overland (Pm)
2488	Overland (Pm)
2489	Overland (Pm)
2490	Overland (Pm)
2491	Overland (Pm)
2492	Overland (Pm)
2493	Overland (Pm)
2494	Overland (Pm)
2495	Overland (Pm)
2496	Overland (Pm)
2497	Overland (Pm)
2498	Overland (Pm)
2499	Overland (Pm)
2500	Overland (Pm)
2501	Overland (Pm)
2502	Overland (Pm)
2503	Overland (Pm)
2504	Overland (Pm)
2505	Overland (Pm)
2506	Overland (Pm)
2507	Overland (Pm)
2508	Overland (Pm)
2509	Overland (Pm)
2510	Overland (Pm)
2511	Overland (Pm)
2512	Overland (Pm)
2513	Overland (Pm)
2514	Overland (Pm)
2515	Overland (Pm)
2516	Overland (Pm)
2517	Overland (Pm)
2518	Overland (Pm)
2519	Overland (Pm)
2520	Overland (Pm)
2521	Overland (Pm)
2522	Overland (Pm)
2523	Overland (Pm)
2524	Overland (Pm)
2525	Overland (Pm)
2526	Overland (Pm)
2527	Overland (Pm)
2528	Overland (Pm)
2529	Overland (Pm)
2530	Overland (Pm)
2531	Overland (Pm)
2532	Overland (Pm)
2533	Overland (Pm)
2534	Overland (Pm)
2535	Overland (Pm)
2536	Overland (Pm)
2537	Overland (Pm)
2538	Overland (Pm)
2539	Overland (Pm)
2540	Overland (Pm)
2541	Overland (Pm)
2542	Overland (Pm)
2543	Overland (Pm)
2544	Overland (Pm)
2545	Overland (Pm)
2546	Overland (Pm)
2547	Overland (Pm)
2548	Overland (Pm)
2549	Overland (Pm)
2550	Overland (Pm)
2551	Overland (Pm)
2552	Overland (Pm)
2553	Overland (Pm)
2554	Overland (Pm)
2555	Overland (Pm)
2556	Overland (Pm)
2557	Overland (Pm)
2558	Overland (Pm)
2559	Overland (Pm)
2560	